

Assignment 8

Aim- Implement the solution for N Queens problem using backtracking

Code

```
#include std <stdio.h>
```

```
int board[20];
```

```
int n;
```

```
int isSafe(int row, int col) {
```

```
    int i, j;
```

```
    for(i=0; i<row; i++) {
```

```
        if((board[i] == col) || (abs(board[i] - col)  
            = abs(i - row))) {
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    return 1;
```

```
}
```

```
void printSolution() {
```

```
    for(int i=0; i<n; i++) {
```

```
        for(int j=0; j<n; j++) {
```

```
            if(board[i] == j)
```

```
                printf("Q ");
```

```
            else
```

```
                printf(". ");
```

```
        } printf("\n");
```

```
    } printf("\n");
```

```
}
```

```

void solve(int row){
    if(row == n){
        printSolution();
        return;
    }
    for(int col = 0; col < n; col++){
        if(isSafe(row, col)){
            board[row] = col;
            solve(row+1);
        }
    }
}

```

```

void main(){
    printf("Enter number of queens: ");
    scanf("%d", &n);
    if(n <= 0 || n > 20){
        printf("Enter N between 1 and 20.\n");
        return;
    }
    solve(0);
}

```

Time Complexity - $O(N!)$
Space Complexity $O(N)$

Obtained solutions

Q	Q
. . Q . .	. . . Q .
. . . Q .	. Q . . .
. Q . . .	 Q
. . . Q .	. . Q . .

$\begin{matrix} Q & . & . & . & . \\ . & . & . & Q & . \\ Q & . & . & . & . \\ . & . & Q & . & . \\ . & . & . & . & Q \end{matrix}$,
 $\begin{matrix} . & Q & . & . & . & . \\ . & . & . & . & Q & . \\ . & . & Q & . & . & . \\ Q & . & . & . & . & . \\ . & . & . & . & . & Q \end{matrix}$,
 $\begin{matrix} . & . & . & . & Q & . \\ . & . & . & . & . & Q \\ . & . & . & . & . & . \\ . & . & . & . & . & . \\ . & . & . & . & . & . \end{matrix}$

$\begin{matrix} . & . & Q & . & . \\ . & . & . & . & Q \\ . & Q & . & . & . \\ . & . & . & . & Q \\ Q & . & . & . & . \end{matrix}$,
 $\begin{matrix} . & . & . & Q & . \\ Q & . & . & . & . \\ . & . & Q & . & . \\ . & . & . & . & Q \\ . & Q & . & . & . \end{matrix}$,
 $\begin{matrix} . & . & . & . & Q & . \\ . & Q & . & . & . & . \\ . & . & . & . & . & Q \\ . & . & . & . & . & . \\ Q & . & . & . & . & . \end{matrix}$

$\begin{matrix} . & . & . & . & Q \\ . & Q & . & . & . \\ . & . & . & Q & . \\ Q & . & . & . & . \\ . & . & Q & . & . \end{matrix}$,
 $\begin{matrix} . & . & . & . & Q \\ . & . & . & Q & . \\ . & . & . & . & . \\ . & . & . & Q & . \\ . & . & . & . & . \end{matrix}$

The first solution is the example to show the backtracking path

Start

- ↳ Place queen at (1, 1)
- ↳ Place queen at (2, 3)
- ↳ Place queen at (3, 5)
- ↳ Place queen at (4, 2)
- ↳ Place queen at (5, 4)